

DBMS Mastery Cheat Sheet

Beginner → Intermediate → Advanced → Expert

A single reference covering everything from core DBMS architecture and ER modeling to normalization, transactions, indexing, and distributed systems — built for coursework, interviews, and the job.

BEGINNER

INTERMEDIATE

ADVANCED

EXPERT

REFERENCE

Covers relational theory · storage & indexing · transactions · distributed & NoSQL systems
(engine-specific notes called out inline where they matter)

Contents

- 01 Beginner — What Is a DBMS & Architecture
- 02 Beginner — Data Models & ER Diagrams
- 03 Beginner — Keys, Constraints & the Relational Model
- 04 Beginner — Normalization (1NF → BCNF)
- 05 Intermediate — Relational Algebra & Schema Mapping
- 06 Intermediate — Transactions & the ACID Model
- 07 Intermediate — Concurrency Control & Isolation Levels
- 08 Advanced — Storage, File Organization & Indexing
- 09 Advanced — Query Processing & Optimization
- 10 Advanced — Recovery, Logging & Failure Handling
- 11 Expert — Distributed Databases, CAP & NoSQL
- 12 Reference — Glossary, Comparisons & Quick Lookup

TIP: DBMS is a theory-heavy subject compared to SQL. Focus on *WHY* each mechanism exists (what problem it solves) rather than memorizing definitions — that's what both interviews and real system design actually test.

A **Database Management System (DBMS)** is software that stores, organizes, secures, and provides controlled access to data, sitting between applications and the raw data files on disk. It replaces ad-hoc file processing with a system that guarantees consistency, reduces redundancy, and supports concurrent multi-user access.

Why not just use files?

File systems	DBMS
Data redundancy across files	Centralized data, redundancy minimized
No enforced consistency	Constraints enforce consistency automatically
Concurrent access is unsafe	Concurrency control built in
No crash recovery	Logging & recovery restore consistent state
Application-level security only	Fine-grained access control & authorization

Three-schema architecture

Level	Describes
External (View) level	What each user/application sees — customized views
Conceptual (Logical) level	The full structure for the whole community of users (tables, relationships)
Internal (Physical) level	How data is actually stored — files, indexes, storage structures

This layering gives **data independence**: you can change how data is physically stored (internal level) without breaking applications built on the conceptual/external levels, and vice versa.

Data independence — two types

Type	Meaning
Physical data independence	Change storage/file structure without changing the conceptual schema
Logical data independence	Change the conceptual schema without changing external views/applications

Core DBMS components

- **Query processor** — parses, optimizes, and executes queries.
- **Storage manager** — manages how data is written to and read from disk.
- **Transaction manager** — ensures ACID properties for concurrent operations.
- **Buffer manager** — caches disk pages in memory to reduce I/O.
- **Catalog / metadata manager** — stores schema, constraints, statistics about the data itself.

DBMS vs RDBMS vs NoSQL (quick framing)

Term	Meaning
DBMS	General term — any system that manages databases (relational or not)
RDBMS	DBMS based on the relational model — tables, rows, SQL, ACID (MySQL, Postgres, Oracle)
NoSQL	Non-relational DBMS families — document, key-value, column, graph (see Section 11)

The **Entity-Relationship (ER) model** is the standard way to design a database at the conceptual level, before writing any SQL. It captures what things exist (entities), what describes them (attributes), and how they relate (relationships).

Building blocks

Element	Meaning	ER diagram symbol
Entity	A real-world object or concept (Student, Order)	Rectangle
Attribute	A property of an entity (name, price)	Oval
Relationship	An association between entities (Enrolls, Places)	Diamond
Primary key attribute	Uniquely identifies an entity instance	Underlined oval
Multivalued attribute	Can hold more than one value	Double oval
Derived attribute	Computed from other attributes (age from DOB)	Dashed oval
Weak entity	Cannot exist without a related "owner" entity	Double rectangle

Relationship cardinality (how many on each side)

Cardinality	Example
One-to-One (1:1)	A Person has exactly one Passport
One-to-Many (1:N)	A Department has many Employees
Many-to-Many (M:N)	Students enroll in many Courses, each Course has many Students

M:N relationships become a separate junction/bridge table when mapped to the relational model — it holds the foreign keys of both sides (e.g., an Enrollment table linking Students and Courses).

Participation & specialization

- **Total (mandatory) participation** — every entity instance **MUST** participate in the relationship (double line in ER diagrams).
- **Partial participation** — participation is optional (single line).
- **Generalization / Specialization (ISA hierarchy)** — modeling subclasses, e.g., Employee generalizes Manager and Engineer, who inherit Employee's attributes.

Mapping ER to relational tables (the core translation rules)

- Each strong entity → its own table, with its attributes as columns and its key as the primary key.
- Each 1:N relationship → foreign key placed on the "many" side's table.
- Each M:N relationship → a new junction table holding both sides' foreign keys (often plus its own attributes).
- Weak entities → table includes the owner's primary key as part of its own primary key (partial key + owner key).
- Multivalued attributes → a separate table (one row per value, referencing the owning entity).

The **relational model** represents data as a set of relations (tables), each a set of tuples (rows) over a set of attributes (columns). Keys and constraints are what keep that data trustworthy.

Types of keys

Key	Definition
Super key	Any set of attributes that uniquely identifies a row
Candidate key	A minimal super key — no attribute can be removed and stay unique
Primary key	The candidate key chosen to identify rows; cannot be NULL
Alternate key	A candidate key NOT chosen as the primary key
Composite key	A key made of two or more attributes together
Foreign key	An attribute (or set) referencing a primary key in another table
Surrogate key	An artificial key (e.g., auto-increment ID) with no business meaning

Integrity constraints

Constraint	Guarantees
Domain constraint	A column's values come from a valid, defined domain/type
Entity integrity	Primary key values are unique and never NULL
Referential integrity	A foreign key value must exist in the referenced table (or be NULL)
Key constraint	Candidate key values are unique across all rows

Referential actions on foreign keys

```
FOREIGN KEY (department_id) REFERENCES departments(department_id)
ON DELETE CASCADE -- delete child rows when parent is deleted
ON DELETE SET NULL -- set FK to NULL when parent is deleted
ON DELETE RESTRICT -- block the delete if child rows exist
ON UPDATE CASCADE -- propagate a key change to child rows
```

Anomalies from poor design (why normalization matters)

Anomaly	Problem it causes
Insertion anomaly	Can't add a fact without also having unrelated data (e.g., can't add a course without a student)
Update anomaly	The same fact stored in multiple places can go out of sync
Deletion anomaly	Deleting one fact accidentally deletes another, unrelated fact

KEY IDEA: Every one of these anomalies traces back to one root cause: a single table trying to represent more than one independent real-world fact at once. Normalization (next section) is the formal fix.

04 Normalization (1NF → BCNF)

BEGINNER

Normalization is the process of organizing tables to reduce redundancy and prevent the anomalies above, by progressively enforcing stricter rules about **functional dependencies** (a rule where one attribute's value determines another's, written $A \rightarrow B$).

Form	Rule	Fixes
1NF	Every column holds atomic (indivisible) values; no repeating groups	Multivalued/composite columns crammed into one field
2NF	1NF + no partial dependency (non-key attribute depends on only PART of a composite key)	Redundancy from composite-key tables
3NF	2NF + no transitive dependency (non-key attribute depends on another non-key attribute)	Redundancy from chains like $A \rightarrow B \rightarrow C$
BCNF	For every dependency $X \rightarrow Y$, X must be a super key	Rare edge cases 3NF misses with overlapping candidate keys

Worked example

```
-- Unnormalized: one row mixes order info + repeated customer info
Orders(order_id, customer_name, customer_city, product, qty, price)

-- Problem: customer_city repeats for every order the same customer makes.
-- Update customer_city in one row and not others -> update anomaly.

-- 3NF fix: split into independent facts
Customers(customer_id PK, customer_name, customer_city)
Products(product_id PK, product_name, price)
Orders(order_id PK, customer_id FK, product_id FK, qty)
```

Denormalization — the deliberate exception

Normalization optimizes for write consistency, not read speed. Data warehouses and reporting systems often **denormalize** on purpose (e.g., star schemas, wide flattened tables) to avoid expensive joins on read-heavy analytical workloads. This is a trade-off, not a mistake — know when each side of it applies.

RULE OF THUMB: OLTP systems (transactional, many small writes) → normalize. OLAP systems (analytical, few large reads) → often denormalize deliberately for query speed.

05 Relational Algebra & Schema Mapping

INTERMEDIATE

Relational algebra is the formal, mathematical foundation SQL is built on. Every SQL query can be expressed as a combination of these operators — knowing them helps you understand what a query optimizer is actually doing under the hood.

Operator	Symbol	SQL equivalent
Selection	σ (sigma)	WHERE
Projection	π (pi)	SELECT (choosing columns)
Union	$\cup$	UNION
Set difference	$-$	EXCEPT / MINUS
Cartesian product	$\times$	CROSS JOIN
Join	$\bowtie$ (bowtie)	JOIN ... ON
Rename	ρ (rho)	AS (alias)

```
-- Relational algebra: find names of employees in the Engineering dept
 $\pi$  name (  $\sigma$  department='Engineering' (Employees) )

-- Equivalent SQL
SELECT name FROM Employees WHERE department = 'Engineering';
```

Schema mapping recap (ER $\rightarrow$ relational $\rightarrow$ physical)

Stage	Output
Conceptual design	ER diagram — entities, relationships, cardinalities
Logical design	Relational schema — tables, keys, normalized to 3NF/BCNF
Physical design	Storage decisions — indexes, partitioning, file organization

This is the same three-schema architecture from Section 01, applied as a design process rather than a runtime structure.

A **transaction** is a sequence of operations treated as one logical unit of work. The DBMS guarantees ACID properties so transactions behave predictably even with crashes or concurrent users.

Property	Guarantee
Atomicity	All operations in the transaction happen, or none do (all-or-nothing)
Consistency	A transaction moves the database from one valid state to another
Isolation	Concurrent transactions don't see each other's uncommitted changes
Durability	Once committed, results survive even a system crash

Transaction states

```
Active -> Partially Committed -> Committed
| |
+-----> Failed -> Aborted
```

State	Meaning
Active	Transaction is executing its operations
Partially committed	Last operation executed, but not yet made permanent
Committed	Transaction succeeded; changes are permanent (durable)
Failed	Normal execution can no longer proceed
Aborted	Transaction rolled back; database restored to pre-transaction state

Schedules — serial vs concurrent

A **schedule** is the order in which operations from multiple transactions are executed. A schedule is **serializable** if it produces the same result as SOME serial (one-at-a-time) execution of the same transactions — this is the gold standard for correctness under concurrency.

Without concurrency control, simultaneous transactions can corrupt each other's results. The DBMS uses protocols — mainly locking or multiversioning — to keep concurrent execution equivalent to some serial order.

Concurrency problems (what can go wrong)

Problem	Description
Dirty read	Reading data written by an uncommitted transaction
Non-repeatable read	Re-reading a row gives a different value within the same transaction
Phantom read	A repeated query returns different ROWS (new/deleted rows) within a transaction
Lost update	Two transactions overwrite each other's changes to the same row

SQL standard isolation levels

Level	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

Higher isolation = stronger correctness guarantees but more locking/blocking overhead and lower concurrency throughput. Pick the lowest level that's still safe for the use case.

Locking protocols

Lock type	Behavior
Shared (S) lock	Multiple transactions can hold it together — for reads
Exclusive (X) lock	Only one transaction can hold it — for writes
Two-Phase Locking (2PL)	Growing phase (acquire locks) then shrinking phase (release) — guarantees serializability
Strict 2PL	All locks held until COMMIT/ROLLBACK — avoids cascading rollbacks

Deadlock

A **deadlock** occurs when two or more transactions each hold a lock the other needs, and each waits forever for the other to release it. DBMSs handle this via **detection** (build a wait-for graph, find cycles, abort one transaction) or **prevention** (e.g., always acquiring locks in a fixed global order).

KEY IDEA: MVCC (Multi-Version Concurrency Control), used by PostgreSQL, MySQL/InnoDB, and Oracle, lets readers see a consistent snapshot without blocking writers — it's why most modern databases don't need heavy read locks for normal SELECT statements.

File organization methods

Method	Description
Heap (unordered) file	Rows stored in insertion order; fast inserts, slow lookups
Sequential (sorted) file	Rows stored sorted by a key; fast range scans, slow inserts
Hash file	Rows placed by a hash of the key; very fast exact-match lookups
Clustered file	Related rows from one or more tables stored physically together

Index structures

Structure	Best for
B-Tree / B+Tree	General purpose — range queries, equality, sorted access (default for most RDBMS)
Hash index	Exact-match equality lookups only; no range queries
Bitmap index	Low-cardinality columns in read-heavy analytical workloads
Clustered index	Physically orders table rows by the index key (one per table)
Non-clustered (secondary) index	A separate structure pointing back to actual row locations

Clustered vs non-clustered — the key distinction

A **clustered index** determines the physical order rows are stored on disk — a table can have only one. A **non-clustered index** is a separate lookup structure with pointers back to the actual rows — a table can have many. This is why primary keys are usually clustered by default, and why clustered index lookups tend to be faster (no extra pointer hop).

Why B+Trees, specifically

- All data lives in leaf nodes at the same depth — lookup time is predictable and logarithmic.
- Leaf nodes are linked together — range scans (BETWEEN, ORDER BY) are fast and sequential.
- High fanout per node matches how disk pages are read — few I/O operations even for huge tables.

The write-side cost of indexes

Every index must be updated on every INSERT/UPDATE/DELETE that touches an indexed column. This is the fundamental read/write trade-off behind “don't just add indexes everywhere” — each one speeds up some reads at the cost of every write.

When a query is submitted, the DBMS doesn't run it literally as written — it transforms it through several stages to find an efficient execution plan.

Query processing pipeline

```
SQL query
-> Parser (checks syntax, builds a parse tree)
-> Query rewriter (view expansion, algebraic simplification)
-> Optimizer (generates & costs multiple execution plans)
-> Execution engine (runs the chosen plan, returns rows)
```

Query optimization approaches

Approach	Idea
Heuristic (rule-based)	Apply known-good transformations, e.g. push selections down before joins
Cost-based	Estimate the I/O/CPU cost of multiple plans using table statistics, pick the cheapest

Most production databases (Postgres, Oracle, SQL Server) use cost-based optimization, relying on statistics (row counts, distributions) that must be kept up to date — stale stats are a common real-world cause of bad execution plans.

Join algorithms

Algorithm	Best when
Nested loop join	One side is small, or an index exists on the join column
Sort-merge join	Both inputs are already sorted (or cheap to sort) on the join key
Hash join	Large, unsorted inputs — build a hash table on the smaller side

Heuristic rules an optimizer applies

- Perform selection (WHERE) and projection (column pruning) as early as possible — shrink data before joining.
- Combine a selection with a following Cartesian product into a proper join.
- Choose the join order that produces the smallest intermediate result sets first.

Recovery mechanisms restore the database to a consistent state after a crash, guaranteeing the **Durability** and **Atomicity** parts of ACID even when the system fails mid-transaction.

Write-Ahead Logging (WAL) — the core mechanism

The golden rule: **log records describing a change must be written to durable storage BEFORE the actual data page is written.** This means the log always has enough information to redo or undo any change, no matter when a crash happens.

Recovery techniques

Technique	How it works
Deferred update	Changes are only written to disk after commit; recovery only needs REDO
Immediate update	Changes may be written before commit; recovery needs both UNDO and REDO
Checkpointing	Periodically flush all committed changes to disk, shrinking the log recovery has to replay
Shadow paging	Write updates to new pages, then atomically swap a page table pointer — avoids UNDO entirely

ARIES-style recovery (used by most modern RDBMS)

1. ANALYSIS -- scan the log to find which transactions were active at crash time
2. REDO -- replay ALL logged changes since the last checkpoint
(even for transactions that later aborted -- redo first, then undo)
3. UNDO -- roll back changes made by transactions that never committed

KEY IDEA: Redoing everything first (even work that will be undone) sounds wasteful, but it's what lets recovery run as a simple, deterministic single pass rather than trying to reason about partial states.

Why distribute a database

- **Scalability** — spread data/load across many machines instead of one bigger machine.
- **Availability** — replicas keep serving traffic if one node fails.
- **Geo-locality** — keep data close to the users/services that need it.

Partitioning (sharding) vs replication

Strategy	What it does
Partitioning / sharding	Splits DIFFERENT data across nodes — scales storage & write throughput
Replication	Copies the SAME data across nodes — scales read throughput & improves availability

The CAP theorem

In a distributed system, during a network partition you must choose between **Consistency** (every read sees the latest write) and **Availability** (every request gets a response) — you cannot have both simultaneously, alongside **Partition tolerance**, which real-world distributed systems must always have.

Choice under partition	Example systems
CP (Consistency + Partition tolerance)	HBase, MongoDB (default config), traditional RDBMS clusters
AP (Availability + Partition tolerance)	Cassandra, DynamoDB, CouchDB

CAP is about behavior specifically DURING a network partition — outside of a partition, most systems try to offer both consistency and availability.

Consistency models

Model	Guarantee
Strong consistency	Every read returns the most recent write, everywhere, immediately
Eventual consistency	All replicas converge to the same value — eventually, if no new writes occur
Causal consistency	Operations that are causally related are seen in the same order by everyone

NoSQL database families

Family	Model	Example systems
Key-Value	Simple key → value pairs	Redis, DynamoDB
Document	Semi-structured JSON/BSON documents	MongoDB, CouchDB
Column-family	Rows with dynamic, sparse columns grouped in families	Cassandra, HBase
Graph	Nodes & edges — optimized for relationship traversal	Neo4j, Amazon Neptune

TIP: Choose relational when data is structured and consistency/joins matter; choose NoSQL when scale, flexible schema, or a specific access pattern (key lookups, graph traversal) dominates the design.

BASE vs ACID

ACID (traditional RDBMS)	BASE (many NoSQL systems)
Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventual consistency
Prioritizes correctness over availability	Prioritizes availability & performance over immediate consistency

OLTP vs OLAP

OLTP	OLAP
Many small, fast read/write transactions	Few large, complex analytical read queries
Normalized schema	Often denormalized (star/snowflake schema)
Row-oriented storage typical	Column-oriented storage typical

Quick glossary

Term	Definition
Schema	The structure/definition of a database — tables, columns, constraints
Instance	The actual data in a database at a given moment in time
View	A saved, virtual table defined by a query — no data stored of its own
Trigger	Code that runs automatically in response to a table event (INSERT/UPDATE/DELETE)
Stored procedure	Precompiled, reusable SQL/procedural code stored in the database
Cursor	A pointer used to iterate row-by-row over a query result in procedural code
Data warehouse	A large, centralized, historical store optimized for analytics
Data mart	A smaller, subject-specific subset of a data warehouse
Sharding key	The column used to decide which partition/shard a row belongs to

Design & exam checklist

- Can you draw the ER diagram before writing any table DDL?
- Is every table at least in 3NF, unless you deliberately denormalized for a reason you can state?
- Do foreign keys have an explicit ON DELETE/ON UPDATE policy, not just a default?
- Have you identified which columns need an index — and accepted the write-cost trade-off?
- For any “what if two users do this at once” question, can you name the concurrency problem AND the isolation level that prevents it?
- For a distributed design, can you say whether it favors C or A during a partition, and why?

You now have the full arc: architecture → ER modeling → keys & normalization → transactions → concurrency → storage & indexing → query optimization → recovery → distributed systems. Pair this with the SQL Mastery Cheat Sheet for the syntax side of the same picture.